介绍

什么是反射

反射是Java的特征之一，是一种间接操作目标对象的机制，核心是JVM在运行状态的时候才动态加载类，对于任意一个类都能够知道这个类所有的属性和方法，并且对于任意一个对象，都能够调用它的方法/访问属性。这种动态获取信息以及动态调用对象方法的功能成为Java语言的反射机制。通过使用反射我们不仅可以获取到任何类的成员方法(Methods)、成员变量(Fields)、构造方法(Constructors)等信息，还可以动态创建Java类实例、调用任意的类方法、修改任意的类成员变量值等。

按照我的理解，java中的反射有点类似与php中实例化另一个文件中的类，并且调用任意的类方法，修改任意的类成员变量值等，就好像的把另一个php代码文件反射到新的代码文件中进行使用。

为什么要用到反射

就像我前面说的，其实从官方定义中就能找到其存在的价值，在运行时获得程序或程序集中每一个类型的成员和成员的信息，从而动态的创建、修改、调用、获取其属性，而不需要事先知道运行的对象是谁。划重点：在运行时而不是编译时。（不改变原有代码逻辑，自行运行的时候动态创建和编译即可）

传统的调用类

我这里先创建一个基础类

```
public class User {
    //成员变量
    public String name="caigo";
    public int age = 18;
    private String gender= "man";
    protected String work= "代码审计";

    //构造方法
    public User(){
        //System.out.println("无参数");
    }

    public User(String name){
        System.out.println("我的名字："+name);
    }

    private User(String name,int age,String gender,String work){
        System.out.println("我的名字是：" + name);
        System.out.println("我的年龄是：" + age);
        System.out.println("我的性别是：" + gender);
        System.out.println("我的工作是：" + work);
    }


    //成员方法
    public void userinfo(String name,int age,String gender,String work){
        this.work=work;
        this.age=age;
        this.name = name;
        this.gender=gender;
```

柴狗安全内部培训资料

```
    }

    protected void users(String name,String gender){
        this.name = name;
        this.gender=gender;
        System.out.println("users成员方法："+name);
        System.out.println("users成员方法："+gender);
    }

}
```

调用

```
// 编译时确定类
public class Test {
    public static void main(String[] args) {

        //无参方法调用
        User Users = new User();

        //有参方法调用
        User Users1 = new User("caigosec");

        User user2 = new User("caigosec", 18, "man", "代码审计");


        //方法调用
        Users.userinfo("caigosec",18,"man","代码审计");
        String name = Users.getName();
        System.out.println(name);

    }
}
```

而反射则打破了这种限制。它提供了一组API，位于 `java.lang.reflect` 包中，主要包括 `Class`、`Constructor`、`Method` 和 `Field` 类。

- `Class` **类：** 反射的入口点，代表一个类或接口。通过 `Class` 对象，你可以获取一个类的所有信息（构造器、方法、字段、注解、父类、实现的接口等）。
- `Constructor` **类：** 代表类的构造方法，用于在运行时创建对象。
- `Method` **类：** 代表类的方法，用于在运行时调用对象的方法。
- `Field` **类：** 代表类的字段（成员变量），用于在运行时访问和修改对象的字段值。

反射使用

Class获取

要使用反射，第一步总是要**获取到目标类的 `Class` 对象。** `Class` 对象是反射的入口。

有四种主要方式可以获取 `Class` 对象：

1. **根据全限定类名获取 (最常用，用于动态加载)：** `Class.forName("包名.类名")`

柴狗安全内部培训资料

```
// 假设有一个 User 类在 org.example.reflectiondemo.test 包中
try {
    Class<?> userClass1 =
Class.forName("org.example.reflectiondemo.test.User");
    System.out.println("通过 Class.forName 获取: " + userClass1.getName());
} catch (ClassNotFoundException e) {
    e.printStackTrace();
}
```

- **优点：** 最灵活，可以在运行时通过字符串动态加载类。
- **缺点：** 需要处理 `ClassNotFoundException`。

2. **根据类名获取 (最直接，用于编译时已知类)：** `类名.class`

```
// 假设 User 类已存在且编译时可知
Class<User> userClass2 = User.class;
System.out.println("通过 .class 关键字获取: " + userClass2.getName());
```

- **优点：** 简单直接，编译时安全，不需要异常处理。
- **缺点：** 只能用于编译时已知的类。

3. **根据对象实例获取：** `对象.getClass()`

```
User user = new User(); // 假设 User 类有无参构造器
Class<?> userClass3 = user.getClass();
System.out.println("通过对象实例获取: " + userClass3.getName());
```

- **优点：** 当你已经有对象实例时最方便。
- **缺点：** 必须先有对象实例。

4. **通过类加载器获取：** `ClassLoader.getSystemClassLoader().loadClass("包名.类名")`

```
ClassLoader classLoader = ClassLoader.getSystemClassLoader();
try {
    Class<?> userClass4 =
classLoader.loadClass("org.example.reflectiondemo.test.User");
    System.out.println("通过类加载器获取: " + userClass4.getName());
} catch (ClassNotFoundException e) {
    e.printStackTrace();
}
```

- **优点：** 提供了对类加载过程更精细的控制，例如可以指定父加载器等。
- **区别于 `Class.forName()`：** `loadClass()` 默认不会执行类的初始化（包括静态代码块的执行），而 `Class.forName()` 默认会初始化类。

Constructor获取

获取 `Constructor` 对象主要有以下几种方式

1、`getConstructor(Class<?>... parameterTypes)`：**获取指定的公共（public）构造方法。**

- **用途：** 当你知道要获取的构造方法的参数类型，并且该构造方法是 `public` 的时候使用。
- **特点：** 只能获取当前类中声明的公共构造方法。

柔狗安全内部培训资料

- **参数：** `parameterTypes` 是一个 `Class` 类型的可变参数列表，代表构造方法的参数顺序和类型。例如，`String.class, int.class` 表示一个接受 `String` 后跟 `int` 参数的构造方法。
- **抛出异常：** 如果找不到匹配的公共构造方法，会抛出 `NoSuchMethodException`

2、`getDeclaredConstructor(Class<?>... parameterTypes)`：**获取当前类声明的指定构造方法（无论访问修饰符）。**

- **用途：** 当你需要获取某个特定参数类型的构造方法，**无论它的访问修饰符是什么**（`public`，`private`，`protected`，默认访问权限），或者该构造方法是私有构造方法时，就应该使用这个方法。
- **特点：** 这个方法**只**会查找**当前类**中直接声明的构造方法，不包括父类的构造方法。
- **参数：** 与 `getConstructor` 相同，是构造方法的参数类型列表。
- **抛出异常：** 如果找不到匹配的构造方法，会抛出 `NoSuchMethodException`

3、`getConstructors()`：**获取所有公共（public）构造方法的数组。**

- **用途：** 当你需要列出类所有可用的公共构造方法时使用。
- **特点：** 返回一个 `Constructor` 对象的数组，只包含当前类中声明的**所有公共构造方法。**
- **返回：** 一个 `Constructor` 数组（如果类没有公共构造方法可能为空）。

4、`getDeclaredConstructors()`：**获取当前类声明的所有构造方法（无论访问修饰符）。**

- **用途：** 当你需要列出当前类**自身**声明的所有构造方法时使用，无论它们的访问修饰符是什么。
- **特点：** 返回一个 `Constructor` 对象的数组，包含当前类中声明的所有构造方法（公共、私有、保护、默认）。
- **返回：** 一个 `Constructor` 数组（如果当前类没有声明任何构造方法可能为空）。

获取到 `Constructor` 对象后，你可以进行以下关键操作：

- `getName()`： 获取构造方法的完整名称（通常就是类的完整名称）。
- `getParameterTypes()`： 获取构造方法的参数类型列表（返回一个 `Class` 数组）。
- `getModifiers()`： 获取构造方法的修饰符（返回一个整数，可以使用 `java.lang.reflect.Modifier` 类来解析，例如判断是否是 `public`、`private` 等）。
- `newInstance(Object... initargs)`：**创建该构造方法对应类的新实例。**
  - `initargs`：传递给构造方法的参数列表。
  - **返回：** 新创建的对象实例。
  - **异常：** 调用时可能抛出 `IllegalAccessException` (没有权限), `IllegalArgumentException` (参数不匹配), `InvocationTargetException` (构造方法内部抛出了异常), `InstantiationException` (类是抽象类或接口，不能实例化)。
- `setAccessible(boolean flag)`：**解除访问限制。**
  - **这是关键！** 如果你要调用**非公共**（`private` 或 `protected` 或默认访问权限）构造方法，你**必须**先调用 `constructor.setAccessible(true)`。这同样是**"暴力反射"**，它绕过了 Java 的访问权限检查。

Method获取

获取 `Method` 对象主要有以下几种方式

某培安全内部培训资料

1、`getMethod(String name, Class<?>... parameterTypes)`：**获取指定的公共（public）方法。**

- **用途：** 当你知道方法的名称和它的参数类型，并且该方法是 `public` 的时候使用。
- **特点：** 这个方法不仅会查找当前类中声明的公共方法，还会查找其**父类**和**实现的接口**中声明的公共方法。
- **抛出异常：** 如果找不到指定的公共方法，会抛出 `NoSuchMethodException`

2、`getDeclaredMethod(String name, Class<?>... parameterTypes)`：**获取当前类声明的指定方法（无论访问修饰符）。**

- **用途：** 当你需要获取某个特定名称和参数类型的方法，**无论它的访问修饰符是什么**（`public`，`private`，`protected`，默认），或者该方法是私有方法时，就应该使用这个方法。
- **特点：** 这个方法**只**会查找**当前类**中直接声明的方法，**不会**查找从父类继承的方法或接口实现的方法。
- **抛出异常：** 如果找不到指定名称的方法，会抛出 `NoSuchMethodException`

3、`getMethods()`：**获取所有公共（public）方法的数组。**

- **用途：** 当你需要列出某个类所有可供外部调用的公共方法时使用。
- **特点：** 返回一个 `Method` 对象的数组，包含当前类声明的所有 `public` 方法、从**父类继承**的所有 `public` 方法以及**实现的接口**中定义的所有 `public` 方法。
- **返回：** 一个 `Method` 数组（如果没有公共方法可能为空）。

4、`getDeclaredMethods()`：**获取当前类声明的所有方法（无论访问修饰符）。**

- **用途：** 当你需要列出**当前类自身**声明的所有方法时使用，无论它们的访问修饰符是什么。
- **特点：** 返回一个 `Method` 对象的数组，**只包含当前类**中声明的方法（公共、私有、保护、默认），**不包含**从父类继承的方法或接口方法。
- **返回：** 一个 `Method` 数组（如果当前类没有声明任何方法可能为空）。

获取到 `Method` 对象后，你可以进行以下关键操作：

- `getName()`： 获取方法的名称。
- `getReturnType()`： 获取方法的返回类型（返回一个 `Class` 对象）。
- `getParameterTypes()`： 获取方法的参数类型列表（返回一个 `Class` 数组）。
- `getModifiers()`： 获取方法的修饰符（返回一个整数，可以使用 `java.lang.reflect.Modifier` 类来解析，例如判断是否是 `public`、`static` 等）。
- `invoke(Object obj, Object... args)`：**调用方法。**
  - `obj`：方法所属的对象实例。如果方法是**静态方法**，则此参数应为 `null`。
  - `args`：传递给方法的参数列表。
  - **返回：** 方法的返回值。如果方法没有返回值（即 `void`），则返回 `null`。
  - **异常：** 调用时可能抛出 `IllegalAccessException` (没有权限) 或 `InvocationTargetException` (被调用的方法内部抛出了异常)。
- `setAccessible(boolean flag)`：**解除访问限制。**
  - **这是关键一步！** 如果你要调用**非公共**（`private` 或 `protected` 或默认访问权限）方法，你**必须**先调用 `method.setAccessible(true)`。这同样是**"暴力反射"**，它绕过了 Java 的访问权限检查。

秦柯安全攻防培训资料

Field获取

主要有以下几种方法来获取 `Field` 对象，每种方法适用于不同的场景：

1、 `getField(String name)`：**获取指定的公共（public）成员变量**

- **用途：** 当你知道字段的名称，并且这个字段是 `public` 的时候使用。
- **特点：** 只能获取当前类及其父类中声明的公共字段。
- **抛出异常：** 如果找不到指定的公共字段，会抛出 `NoSuchFieldException`

2、 `getDeclaredField(String name)`：**获取指定的所有（包括私有、保护、默认访问权限的）成员变量**

- **用途：** 当你需要获取某个特定名称的字段，无论其访问修饰符是什么（`public`，`private`，`protected`，默认），或者该字段是私有字段时，就应该使用这个方法。
- **特点：** 只能获取**当前类**中声明的字段，**不会**获取其父类中声明的字段。
- **抛出异常：** 如果找不到指定名称的字段，会抛出 `NoSuchFieldException`。

`getFields()`：**获取所有公共（public）成员变量的数组**

- **用途：** 当你需要遍历一个类的所有公共字段时使用。
- **特点：** 返回一个 `Field` 对象的数组，包含当前类及其所有父类中声明的**所有公共字段**。
- **返回空数组：** 如果没有公共字段，返回一个长度为零的数组。

`getDeclaredFields()`：**获取当前类中声明的所有（包括私有、保护、默认访问权限的）成员变量的数组**

- **用途：** 当你需要遍历一个类**自身**声明的所有字段时使用，无论它们的访问修饰符是什么。
- **特点：** 返回一个 `Field` 对象的数组，只包含**当前类**中声明的字段，**不包含其父类**中声明的字段。
- **返回空数组：** 如果当前类没有声明任何字段，返回一个长度为零的数组。

获取到 `Field` 对象后，你可以进行以下操作：

- `getName()`： 获取字段的名称。
- `getType()`： 获取字段的类型（返回一个 `Class` 对象）。
- `getModifiers()`： 获取字段的修饰符（返回一个整数，可以使用 `java.lang.reflect.Modifier` 类来解析）。
- `get(Object obj)`： 获取指定对象上该字段的值。
  - 如果字段是静态的，`obj` 参数可以传 `null`。
  - 如果字段是私有的，需要先调用 `field.setAccessible(true)`。
- `set(Object obj, Object value)`： 设置指定对象上该字段的值。
  - 如果字段是静态的，`obj` 参数可以传 `null`。
  - 如果字段是私有的，需要先调用 `field.setAccessible(true)`。
- `setAccessible(boolean flag)`： 这个是**关键**！如果你想访问或修改非公共（`private` 或 `protected` 或默认访问权限）字段，你必须先调用 `field.setAccessible(true)`。这被称为“**暴力反射**”，因为它绕过了 Java 的访问权限检查。

老狗安全内部培训资料

实际调用案例

```
public class Reflection {
    public static void main(String[] args) throws ClassNotFoundException,
NoSuchFieldException, IllegalAccessException, NoSuchMethodException,
InvocationTargetException, InstantiationException, InvocationTargetException,
IOException {
    //正常调用命令执行
        Runtime.getRuntime().exec("calc");
    //反射调用命令执行
        Class<?> aClass = Class.forName("java.lang.Runtime");
        Constructor<?> declaredConstructor = aClass.getDeclaredConstructor();
        declaredConstructor.setAccessible(true);

        Method exec = aClass.getMethod("exec", String.class);
        Method getRuntime = aClass.getMethod("getRuntime");

        Object invoke = getRuntime.invoke(aClass);
        exec.invoke(invoke,"calc");

        Class<?> aClass1 = Class.forName("java.lang.Runtime");
        Constructor<?> constructor = aClass1.getDeclaredConstructor();
        constructor.setAccessible(true);
        Method exec = aClass1.getMethod("exec", String.class);
        exec.invoke(constructor.newInstance(),"calc");
    }


}
```

若依不安全的反射分析

执行点

```
public static void invokeMethod(SysJob sysJob) throws Exception
{
    // 1. 获取调用目标字符串
    String invokeTarget = sysJob.getInvokeTarget();

    // 2. 解析 Bean 名称
    String beanName = getBeanName(invokeTarget);

    // 3. 解析方法名称
    String methodName = getMethodName(invokeTarget);

    // 4. 解析方法参数
    List<Object[]> methodParams = getMethodParams(invokeTarget);

    // 5. 判断是 Spring Bean 还是普通类
    if (!isValidClassName(beanName)) // 如果 beanName 不是一个有效的类名（通常意味着它
是Spring Bean的名称）
    {
        // 5.1 从 Spring 容器获取 Bean 实例
        Object bean = SpringUtils.getBean(beanName);
```

柴狗安全内部培训资料

```
        // 5.2 动态调用方法
        invokeMethod(bean, methodName, methodParams);
    }
    else // 如果 beanName 是一个有效的类名（全限定名）
    {
        // 5.3 通过反射创建类实例 (调用无参构造器)
        Object bean = Class.forName(beanName).newInstance(); // 注意：
newInstance() 只能调用无参公共构造器
        // 5.4 动态调用方法
        invokeMethod(bean, methodName, methodParams);
    }
}
```

关键处理

```
private static void invokeMethod(Object bean, String methodName, List<Object[]>
methodParams)
        throws NoSuchMethodException, SecurityException, IllegalAccessException,
IllegalArgumentException,
        InvocationTargetException
{
    // 1. 检查方法参数是否存在且不为空
    if (StringUtils.isNotNull(methodParams) && methodParams.size() > 0)
    {
        // 1.1 获取带参数的方法对象
        // getMethodParamsType(methodParams) 应该是一个辅助方法，用于从
List<Object[]> 中提取 Class[] (参数类型数组)
        Method method = bean.getClass().getDeclaredMethod(methodName,
getMethodParamsType(methodParams));

        // 1.2 调用带参数的方法
        // getMethodParamsValue(methodParams) 应该是一个辅助方法，用于从
List<Object[]> 中提取 Object[] (参数值数组)
        method.invoke(bean, getMethodParamsValue(methodParams));
    }
    else // 如果 methodParams 为空或为 null
    {
        // 2. 获取无参数的方法对象
        Method method = bean.getClass().getDeclaredMethod(methodName);

        // 2.1 调用无参数方法
        method.invoke(bean);
    }
}
```

柔韧至全内部培训资料